

Guaranteed Step Accuracy

Combining Agile decomposition and test-driven development in LLM software delivery



Ed Barlow Web Cloud Studio July 2026

Guaranteed Step Accuracy

Ed Barlow — Web Cloud Studio

Abstract

LLM builds fail multiplicatively. A build is a chain of steps; unverified error in one step propagates into every step that consumes its output. Drydock [1] applies the two quality processes the industry already validated. Agile decomposition reduces the Epic to stories, each with acceptance criteria authored before any code exists. Test-driven development makes those criteria the Definition of Done: executed after each build step as deterministic, non-agentic checks that consume no model context and cannot be self-reported by the model under evaluation. A story verifies only when its acceptance passes. Verification gates the dependency graph, so error cannot cross a checked boundary. Chain accuracy stops decaying with build length. A practical side effect follows: bounded steps with external verification need less model capability, so smaller models complete builds that otherwise require frontier models.

Keywords: test-driven development, acceptance criteria, Agile, LLM code generation, verification, definition of done, step accuracy, model scaling

1. The Arithmetic

A build of n sequential steps, each correct with probability p , is correct with probability p^n . At $p = 0.95$ and $n = 40$, that is under 13%. This arithmetic — not model quality — is why long agentic builds drift and why regenerating an application from the same inputs rarely yields the same working software.

The structure has two defects: steps are too large to verify, and verification, where it exists, is performed by the model that did the work. The industry solved both for human developers. Agile decomposes work into stories with acceptance criteria agreed before implementation [3]. TDD writes the test before the code, so "done" is an executable fact rather than an author's claim [4]. Drydock transfers both processes to LLM delivery intact. A companion paper [2] covers the delivery-optimization consequences; this paper covers correctness.

The claim: **when every atomic story carries acceptance criteria declared before its build and verified deterministically after it, error cannot cross a graph boundary, and build correctness becomes independent of build length.** Verified steps reset the chain. p^n becomes p per step, enforced n times.

2. Agile Decomposition: Stories That Carry Acceptance

`drydock analyze` decomposes the imported specification — the Epic — into features and stories. Every story acquires acceptance criteria at decomposition time, not build time [1, §"SAIL Phase 2"]. Three mechanisms do quality work usually left to hope:

Blockers are first-class. Ambiguity produces `BLOCKERS.md` and structured questionnaires, not guesses. The product owner — Drydock's Commander — answers; analysis reruns; the cycle repeats until the plan reports `Ready`. A human decision resolves every ambiguity before it can become code. This is the Agile refinement loop, run between a human and an LLM.

Spikes separate research from construction. Genuine unknowns become spike blocks whose product is a recorded `finding:`. Uncertainty is retired once, in writing.

Stories are atomic. Decomposition continues until each story's context fits a bounded token budget and its behavior is small enough to state acceptance for. A story that cannot be given honest acceptance criteria is not yet decomposed.

The output is the Manifest: a dependency graph in which every node carries its own verification contract. A story runs only when its dependencies are `closed/verified`. No step builds on unverified output [1, `$"Execution Rules"`].

3. The TDD Inversion: Acceptance Before Build

Every typed specification file ends with three sections: **Programmatic Acceptance**, **User Acceptance**, and **Guardrails** [1, `$"Specification File Format"`]. All three are authored during planning and are human-owned. The build may add finer tests; it may never remove or weaken a declared assertion.

Programmatic Acceptance is the Definition of Done. Each check is a stable heading, an intent statement, and an executable Python assertion. The story and the tests that prove it are written in the same build step. The criteria exist first; the build makes them pass.

Verification is deterministic and non-agentic. Declared checks run as post-build hooks — plain Python, outside any model context. Two consequences: verification consumes no tokens and cannot degrade under context pressure, and the model cannot self-report success. An agent asked "did you finish?" gives a sincere, unreliable answer — the failure TDD was invented to prevent in humans. Drydock never asks. The check passes or the story is `closed/failed`.

Guardrails are permanent negative assertions. Acceptance states what the software must do; guardrails state what it must never do. They guard against model hallucination, not specification omission. Example: `DATABASE.md` mandates that all data access flow through a typed class library; a review that finds raw SQL outside the encapsulation boundary fails [1, `$"Database Encapsulation"`]. Guardrails persist across rebuilds.

User Acceptance covers the honest remainder. Checks that cannot be truthfully automated — visual quality, workflow feel — surface to the Commander with build evidence in the QuarterDeck review console. They are declared, not skipped and not dishonestly mechanized.

4. The Verified State Machine

Every block moves through one state machine [1, `$"drydock build — PseudoCode State Machine"`]:

Transition	Rule
<code>pending → closed/verified</code>	Build agent succeeds, files are written, and programmatic acceptance passes. All three. No override.
<code>pending → closed/failed</code>	Build fails, or build succeeds and acceptance fails. The failure reason is recorded as a one-line <code>finding:</code> .
Frontier admission	Only blocks whose external dependencies are <code>closed/verified</code> may run. Failure is containing: downstream work never starts on a failed foundation.
<code>closed/failed → pending</code>	The Commander reopens failed work from the QuarterDeck, revising instructions, acceptance, or scope. The revision is recorded. No silent retry.

Every completed block writes evidence to a per-block file. Material decisions append to the Ship's Log. The system produces verified software and a verifiable record: what was accepted, by which check, on which evidence, decided by whom.

5. Why Step Accuracy Is Guaranteed

Drydock changes both parameters of the $\$1$ arithmetic.

It raises p . Each step is atomic. Its context is bounded and complete: the files it implements, compacted interfaces of what it consumes, the Commander's standing intent. Its success condition appears in the prompt as executable criteria. Small, fully specified, criterion-anchored tasks are the regime where model reliability is highest.

It removes the exponent. Acceptance runs deterministically at every graph edge. A defect is detected at the step that created it, where the fix is local — rerun one story. The chain is only ever one unverified step long.

The scope of the guarantee is exact: programmatic acceptance guarantees the software satisfies its declared criteria. Criteria quality remains a human responsibility — which is where Agile puts it: with the product owner, at refinement time, backed by an analysis phase that forces ambiguity out before construction.

6. Side Effect: Smaller Models Suffice

Long-horizon agentic coding requires frontier models because the model must survive an unverified chain: retain constraints across a large context, notice its own errors, and self-correct without external signal. Drydock externalizes all three into process:

Demand on the model	Replaced by
Context retention	Deterministic prompt assembly from typed files
Error detection	Post-build acceptance checks
Recovery	The state machine and Commander review

What remains is the task mid-tier and frontier models perform comparably well: implement one bounded story against explicit criteria. Prompts are smaller [2], verification is free (it is non-agentic and consumes no model context), and the correctness guarantee never depended on the model's self-assessment. Teams reserve frontier models for analysis and planning — where judgment density is highest — and delegate story implementation to cheaper models. Verification effort, not model scale, carries the quality.

7. Related Work

TDD [4] and Agile refinement [3] map here to concrete mechanisms: Programmatic Acceptance sections, the verified state machine, Manifest story blocks, the blocker loop. Spec Kit [5] established the specification-and-task-list agent interface; Drydock stories are enriched Spec Kit tasks carrying states, dependencies, and acceptance. Contemporary agent frameworks evaluate completion by model self-report or LLM-as-judge; Drydock's position is that deterministic, non-agentic verification is the only evaluation that composes across a long build. The author knows no other published system that combines pre-declared, human-owned acceptance criteria with graph-gated deterministic verification as the delivery mechanism for specification-driven LLM builds.

8. Conclusion

Decompose until stories are small enough to state acceptance for. Declare done before building. Let executable tests — not the author — decide. LLMs need this discipline more than humans did. Drydock enforces it: acceptance-carrying atomic stories, a graph that admits no step atop unverified work, and deterministic checks the model can neither see nor game. Step accuracy holds at every boundary. Correctness stops decaying with build length. The model shrinks to fit the step. The result is working software you can rebuild, and trust, on demand.

References

- [1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery*. Web Cloud Studio, 2026. <https://github.com/webcloudstudio/Drydock>
- [2] E. Barlow. *Optimizing Specification-Driven Delivery: Atomic Decomposition, Build Graphs, and Context Engineering for Reproducible LLM Software Delivery*. Web Cloud Studio, 2026.
- [3] K. Beck et al. *Manifesto for Agile Software Development*. 2001. <https://agilemanifesto.org>
- [4] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [5] GitHub. *Spec Kit: Toolkit for Spec-Driven Development*. 2025. <https://github.com/github/spec-kit>